AccessPath AI

"Don't just test a website. Test whether a real person can actually complete the task."

- Main Objective – Task-Level Accessibility & Usability Testing with Live AI Browser Agent
- Hackathon – HackIIITD 2026 — Indraprastha Institute of Information Technology Delhi (IIIT-D)
- Track / Theme – AI Browser Agents & Public Infrastructure Usability (TinyFish Powered)
- PS Category – Software / Web Accessibility / Autonomous Testing
- **Team Name (Registered on portal) – [Team Name Placeholder]**
- **Team Members – [Member 1, Member 2, Member 3, Member 4 Placeholder]**

SECTION 01

1 Idea / Approach Detail

Problem

> Websites often pass technical accessibility checklists yet remain completely unusable for real people. While static scanners verify HTML tags and contrast, they cannot verify whether a real person can complete end-to-end tasks like submitting a scholarship or uploading required documents.

> Organizations need an automated, task-first testing platform that detects where and why users get stuck across dynamic forms, mobile viewports, and regional languages.

Solution

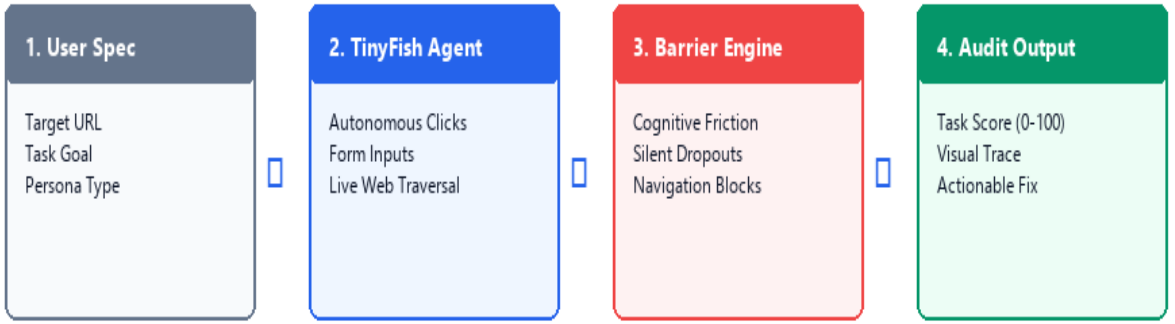
> AccessPath AI deploys a live AI browser agent to autonomously attempt real tasks on live portals from diverse user perspectives (mobile-first, Hindi-speaking, low digital-literacy).

> When a user encounters a friction point or unrecoverable barrier, the system captures step evidence, computes a Task Success Score (0–100), and provides actionable engineering fixes.

What Makes It Different

> Task-First Usability Over Static Compliance
Unlike static WCAG scanners that inspect code in isolation, AccessPath AI evaluates the complete multi-step journey on the live website.

> Persona-Aware Execution: Define ➔ Execute ➔ Detect ➔ Improve
Simulates realistic personas (device constraints, language hurdles, low literacy) to uncover hidden UX dead ends that traditional linters completely miss.



SECTION 02

2 Technical Approach

PROCESS & ARCHITECTURAL PIPELINE:

➢ 1. Test Specification Layer: Accepts Target URL, Goal Task ('Apply for Scholarship'), and User Persona profile (Mobile, Hindi, First-Time User).

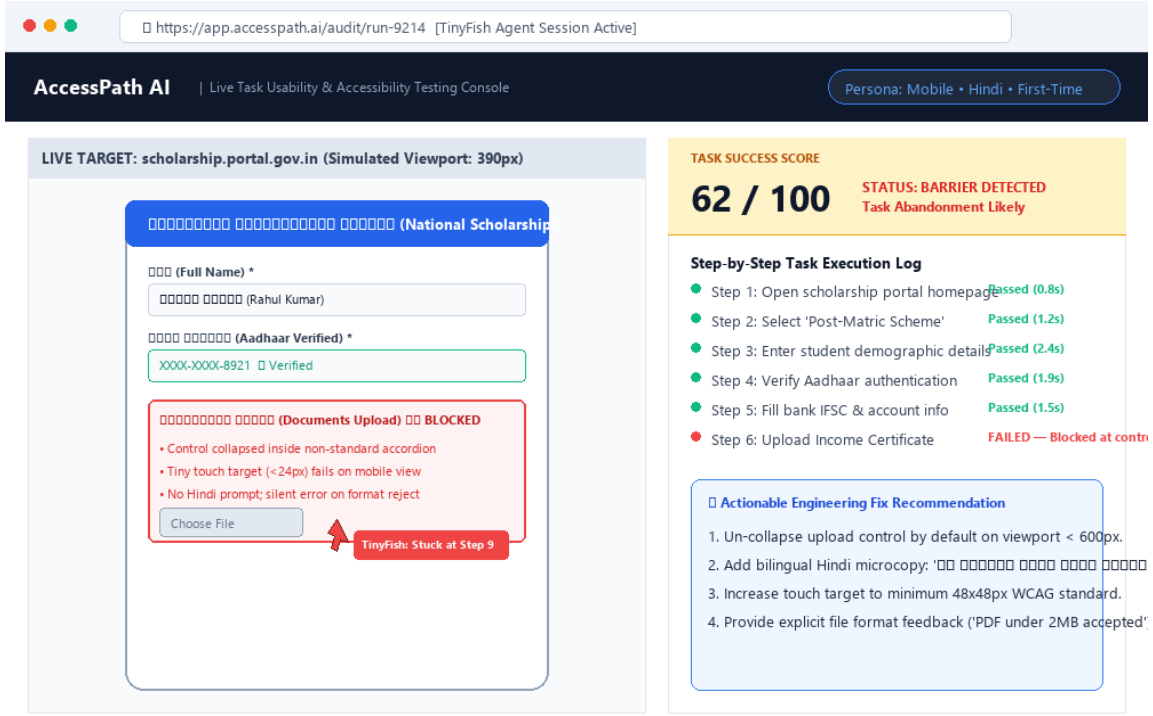
➢ 2. Live Web Agent Layer (TinyFish): Spawns autonomous browser session executing real clicks, input typing, dropdown selects, and dynamic modal navigation.

➢ 3. Observation & Barrier Detection: Monitors execution telemetry, detecting DOM dead ends, cognitive friction, confusing controls, and silent form rejections.

➢ 4. Audit & Diagnostic Engine: Computes Task Success Score (0-100), captures failure screenshots, and outputs engineering remediation guidance.

User Input ➔ TinyFish Agent ➔ Live Portal ➔ Friction Detection ➔ Task Score & Fix

Prototype :



TECH STACK:

Web Agent: TinyFish Web Agent API • Frontend: React.js + TypeScript + TailwindCSS • Backend: Python + FastAPI • Analysis: Vision LLM Reasoner

SECTION 03

3 Impact and Benefits

Social & Citizen Impact

> Equitable Public Services: Ensures students and rural citizens can successfully complete scholarship, pension, and welfare applications without agent exploitation.

> Overcoming Digital Literacy Barriers: Detects UI jargon and language hurdles that cause first-time users to abandon critical government portals.

> Assistive Dignity: Verifies functional completion for screen-reader and mobile users beyond surface-level tag compliance.

Economic & Operational Impact

> Dramatically Cheaper Usability Audits: Automates multi-persona user testing that traditionally costs thousands of dollars and weeks of manual user research.

> Reduced Support Desk Ticket Volume: Eliminating submission drop-offs cuts grievance tickets, helpline congestion, and manual document re-submissions.

> Increased Form Completion Rates: Organizations capture higher digital service adoption and revenue throughput across public and commercial portals.

Target Sectors & Deployment

> Government Public Portals: National & State scholarship portals, Direct Benefit Transfer (DBT), ration card and revenue services.

> Universities & Colleges: Admissions, entrance test registration, fee payment, and examination portals.

> Banking & FinTech: Digital KYC onboarding, micro-credit applications, insurance claim submission flows.

> Healthcare & Telemedicine: Appointment booking, diagnostic report retrieval, and health scheme registrations.

SECTION 04

4 TinyFish Integration & Technical Feasibility

Potential Challenges

➢ Dynamic Client-Rendered SPAs & Popups
Modern portals rely heavily on React/Vue, asynchronous validation, modal popups, and anti-bot checks that break basic headless scrapers.

➢ Subjective Cognitive Usability vs Bugs
Distinguishing between a broken HTML element and a confusing UX layout (e.g. buried upload button) requires reasoning, not just DOM parsing.

➢ Execution Latency & Site Flakiness
Live multi-step web browsing can encounter slow government servers, dynamic reloads, or temporary network timeouts.

Mitigation & TinyFish Division

➢ TinyFish as Web Interaction Layer
TinyFish serves as the autonomous 'eyes and hands'—handling live browser control, DOM interactions, resilient retries, and multi-page flows seamlessly.

➢ AccessPath as Cognitive Analysis Layer
AccessPath evaluates task intent, scores cognitive friction, and explains exactly why the user journey failed with visual evidence.

➢ Graceful Degradation & Step Checkpoints
System records deterministic telemetry at each step; if a live network stall occurs, prior completed steps remain cached and auditable.

TINYFISH WEB AGENT • COGNITIVE BARRIER DETECTION • FAIL-SOFT AUDIT ARCHITECTURE

SECTION 05

5 Research and References

1. World Wide Web Consortium (W3C), “Web Content Accessibility Guidelines (WCAG) 2.2,” W3C Recommendation, Oct 2023.
   https://www.w3.org/TR/WCAG22/

2. Ministry of Electronics and Information Technology (MeitY), Govt. of India, “Guidelines for Indian Government Websites (GIGW 3.0),” 2023.
   https://guidelines.gov.in/

3. WebAIM, “The WebAIM Million: An annual accessibility analysis of the top 1,000,000 home pages,” WebAIM Research Report, 2024.
   https://webaim.org/projects/million/

4. Zhou S. et al., “WebArena: A Realistic Web Environment for Building Autonomous Agents,” International Conference on Learning Representations (ICLR), 2024.
   https://arxiv.org/abs/2307.13854

5. Gur I. et al., “A Real-World WebAgent with Planning, Long Context Understanding, and HTML Pruning,” Advances in Neural Information Processing Systems (NeurIPS), 2023.
   https://arxiv.org/abs/2307.12856

6. Power C. et al., “Guidelines Are Only Half of the Story: How Accessibility Guidelines Fail People with Disabilities,” ACM CHI Conference on Human Factors in Computing Systems, 2012.
   https://dl.acm.org/doi/10.1145/2207676.2207736

7. National e-Governance Division (NeGD), “Digital India Usability and Citizen Interface Standards,” MeitY, 2022.
   https://www.digitalindia.gov.in/

SECTION 06

6 Solution Summary

Problem ➔

Websites can be technically compliant under static WCAG rules and yet remain impossible for citizens to use. Static code checkers only inspect isolated syntax tags, missing end-to-end user friction, mobile viewport traps, and Hindi language barriers on real portals.

Proposed Solution ➔

AccessPath AI pairs a live AI browser agent (powered by TinyFish) with task-level cognitive diagnostics. It tests live websites from specific user persona perspectives, executing the full workflow to verify whether a real person can actually achieve their goal.

Novelty / USP ➔

Task-first outcome testing rather than page-level checklists. Provides persona-aware simulation, live web multi-step execution, and an objective Task Success Score (0-100) with visual evidence and engineering remediation.

Feasibility ➔

Feasible 24-hour hackathon prototype combining TinyFish Web Agent API for autonomous browsing, React/Tailwind for the audit console, and Python/LLM reasoning for barrier scoring and recommendations.

Impact ➔

CITIZEN ACCESS: Enables students and rural users to complete critical welfare and scholarship forms.
ORGANIZATIONAL EFFICIENCY: Lowers usability testing costs and prevents grievance ticket backlogs.
OUTCOME ACCOUNTABILITY: Moves web accessibility evaluation from passive checklists to proven task completion.

SECTION 07

7 The Big Idea: Accessibility Measured by Outcomes

"Traditional Testing Asks: 'Does the website pass a code checklist?'

AccessPath AI Asks: 'Can a real human being actually complete the task?'"

AccessPath AI helps organizations build websites that people can actually use — not just websites that pass a checklist.

SUBMISSION & TEAM DETAILS

- Hackathon Event: HackIIITD 2026 — IIIT Delhi (TinyFish Web Agent Track)
- Project Name: AccessPath AI
- Team Name (Registered): _____ [Space for Team Name]
- Team Leader: _____ [Name, Email / Contact]
- Team Members: 1. _____
2. _____
3. _____
- GitHub Repository: https://github.com/_____ [Prototype Repo]